

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CONCEPT MAPPING

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 2 – Concept Mapping
Weightage:	20% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO1 Statement:	<i>Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)</i>		
Student Name:	P.Hemika	Reg. No.:	192424259
Section / Batch:	AI & DS	Date:	23-07-2026

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Draw a clear and neat concept map for each question.
- Identify all key concepts and arrange them in a logical hierarchy.
- Show meaningful linkages between concepts using arrows and connecting words.
- Compare related concepts wherever required and justify the relationships clearly.
- Ensure the concept map is well-organized, readable, and properly labelled.

Question Type	Focus Area	Questions	Marks
Concept Mapping	Map algorithm efficiency concepts using asymptotic analysis, search techniques, recurrence relations, and recursion tree method	Q1 – Q2	Q1 –50 Q2 –50
GRAND TOTAL:		100 Marks	

SECTION A – CONCEPT MAPPING

Code of Conduct

I P.Hemika 192424259 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Hemika.P

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

Marks Evaluation:

Question No.	Concept Identification (25%)	Linkages (25%)	Organization (20%)	Integration of Literature (20%)	Clarity (10%)	Total Marks (Each – 50)
Q1						
Q2						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Concept Mapping

Q37. Consider the concept map: Algorithm → Growth Function → Big-Omega → Lower Bound →

Best-case Explain how lower bound represents minimum execution time. Extend the map with performance

guarantee node. Justify why best-case analysis is useful. Interpret how this contributes to overall performance understanding.

Answer:-

Explanation

A growth function describes how the running time of an algorithm changes as the input size (n) increases.

Big-Omega (Ω) represents the lower bound of an algorithm. It shows the minimum amount of time the algorithm will take for an input of size n . In other words, the algorithm cannot perform faster than this bound.

Lower Bound Represents Minimum Execution Time

A lower bound guarantees that the algorithm requires at least $\Omega(f(n))$ time, even in the most favorable situation. Therefore, it represents the minimum execution time (best-case performance).

Extended Concept Map

Algorithm → Growth Function → Big-Omega (Ω) → Lower Bound → Best-case → Performance Guarantee

Performance Guarantee: Ensures that the algorithm will take at least $\Omega(f(n))$ time for any input.

Why Best-case Analysis is Useful

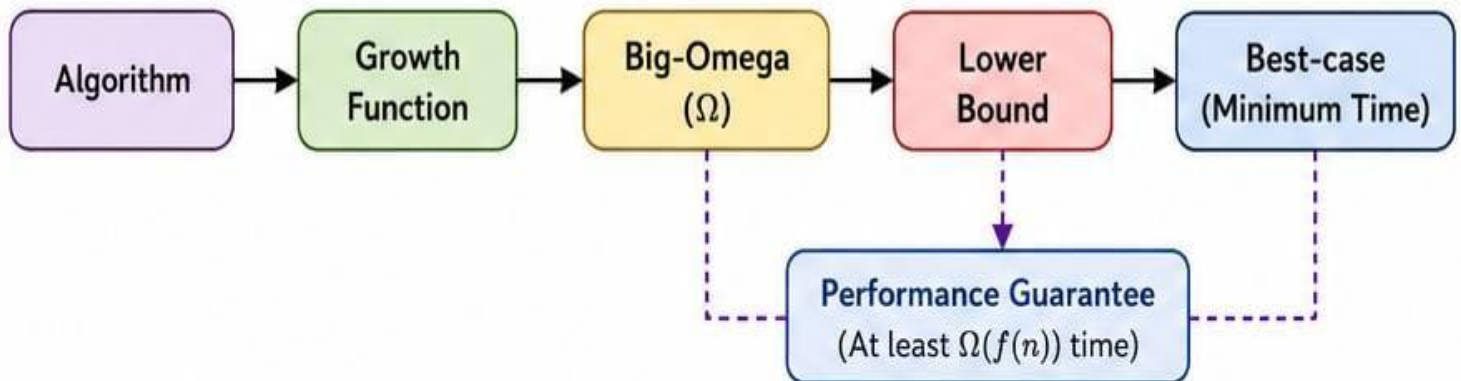
Shows the minimum time required by the algorithm.

Helps understand the most efficient possible execution.

Useful for comparing algorithms under ideal conditions.

Interpretation

Best-case analysis provides the minimum performance limit of an algorithm. When combined with average-case and worst-case analyses, it gives a complete understanding of the algorithm's efficiency.



- **Algorithm:** A problem-solving procedure.
- **Growth Function:** Describes how time/space grows with input size (n).
- **Big-Omega (Ω):** Provides a lower bound on the growth function.
- **Lower Bound:** Minimum time taken by the algorithm in any case.
- **Best-case:** Best possible input scenario with minimum execution time.
- **Performance Guarantee:** Assurance that algorithm will take at least this much time.

Q38. Consider the concept map: Algorithm → Growth Function → Big-Theta → Tight Bound → Exact

Complexity Explain how tight bound represents exact growth. Extend the map by linking Big-O and Big-Omega.

Justify why Big-Theta is preferred for precise analysis. Interpret how this improves decision making.

Explanation

A growth function shows how an algorithm's running time increases with input size.

Big-Theta (Θ) represents the tight bound of an algorithm. It provides the exact order of growth, meaning the running time is bounded by both an upper limit and a lower limit.

Tight Bound Represents Exact Growth

A tight bound means the running time grows at $\Theta(f(n))$, where:

Upper Bound: $O(f(n))$

Lower Bound: $\Omega(f(n))$

Since both bounds are the same, $\Theta(f(n)) = O(f(n)) \cap \Omega(f(n))$, giving the exact complexity.

Extended Concept Map

Algorithm $\rightarrow$ Growth Function $\rightarrow$ Big-Theta (Θ) $\rightarrow$ Tight Bound $\rightarrow$ Exact Complexity

Link:

Big-O (Upper Bound)

Big-Omega (Lower Bound)

Big-Theta = $O(f(n)) \cap \Omega(f(n))$

Why Big-Theta is Preferred

Gives the exact growth rate of an algorithm.

More precise than using only Big-O or Big-Omega.

Helps compare algorithms accurately.

Interpretation

Big-Theta provides the most accurate measure of algorithm performance because it considers both upper and lower bounds. This helps developers choose the most efficient algorithm and make better design decisions.

